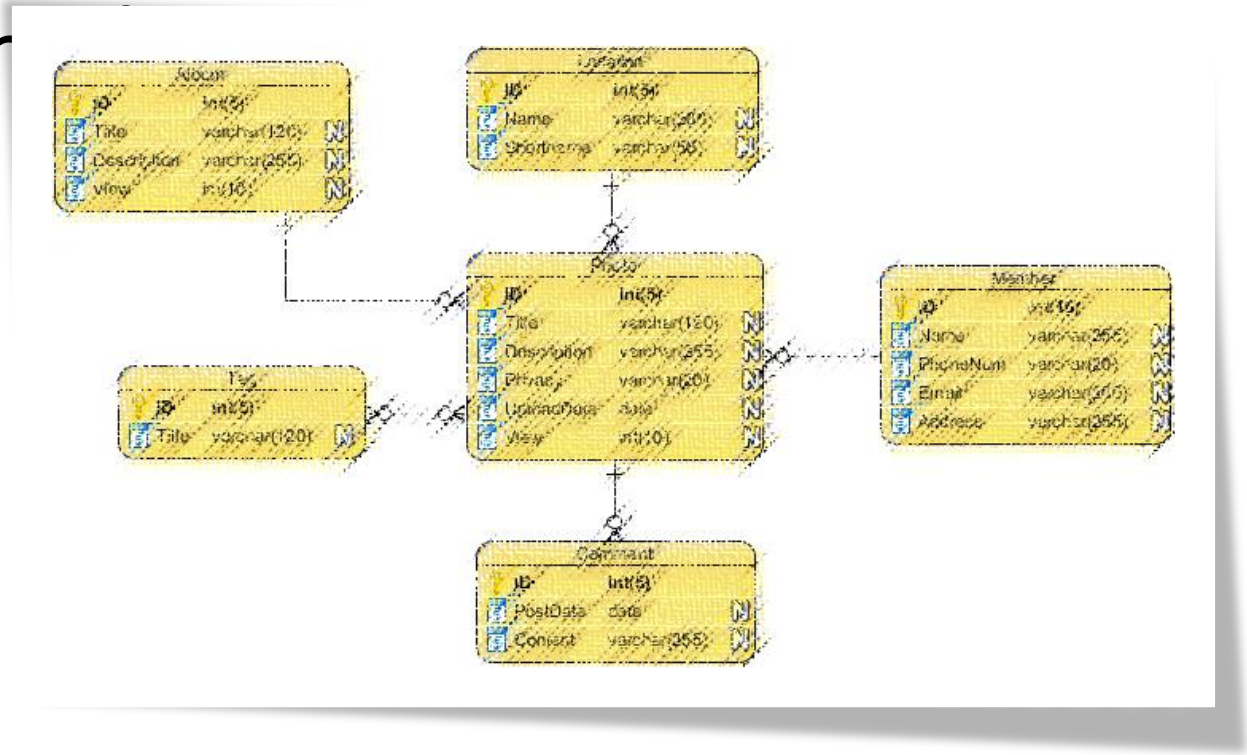
Database Denorm

SWE 4601 Software Design and Architecture



Normalization & Denormalization

- **Normalization** is a technique of eliminating the redundant data from the database.
- **Denormalization** is the inverse process of normalization where the redundancy is added to the data to improve the performance of the specific application.
- By updating, inserting, and deleting records through foreign key relationships, normalization reduces instances of missed or orphaned records in your database.
- Number of tables Increases in normalization

  Decreases in denormalization
- While normalized data is optimized for entity level transactions, denormalized data is optimized for answering business questions and driving decision making.

Pros/ advantage of Normalization

- Complies to ACID property
  - Atomicity
  - Consistency
  - Isolation
  - Durability
- Updates run quickly
- Inserts run quickly
- Clear understanding of data
- No redundancy

So, why denormalization?

Normalized DB require join

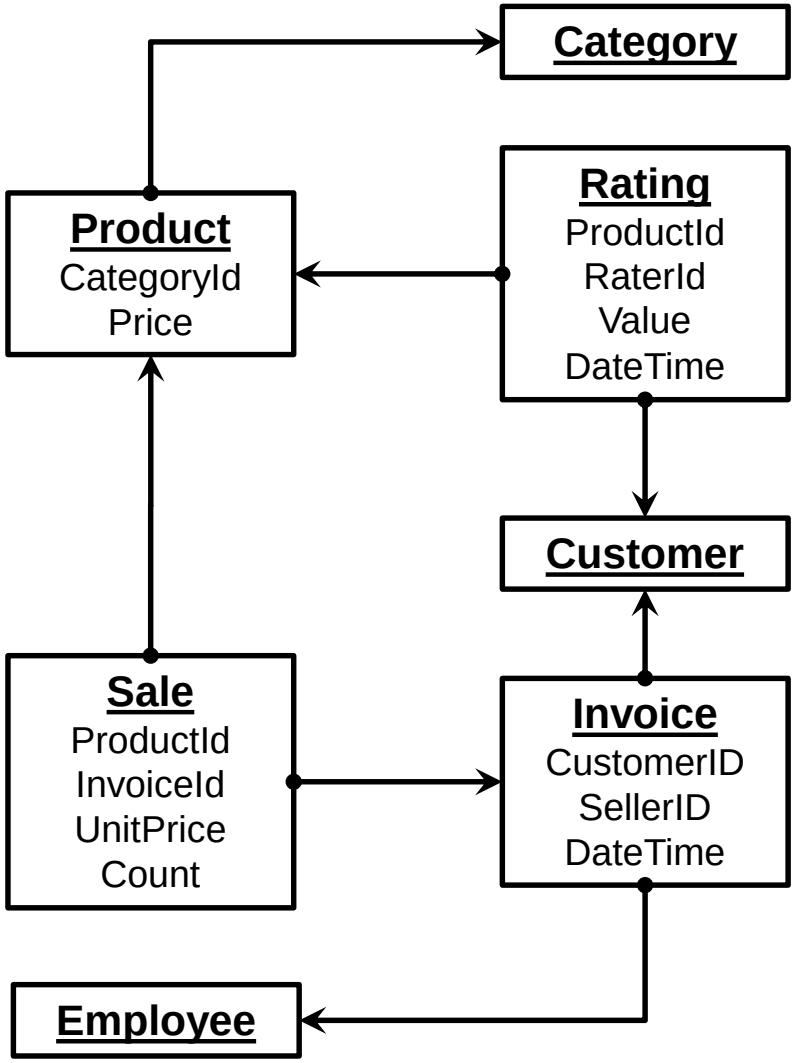
A lot of join

May be crazy lot of join

why → when join is expensive and queries need a lot of joins

when → normalization design is done as early as possible, but denormalization is done on demand basis

The kid's shop database

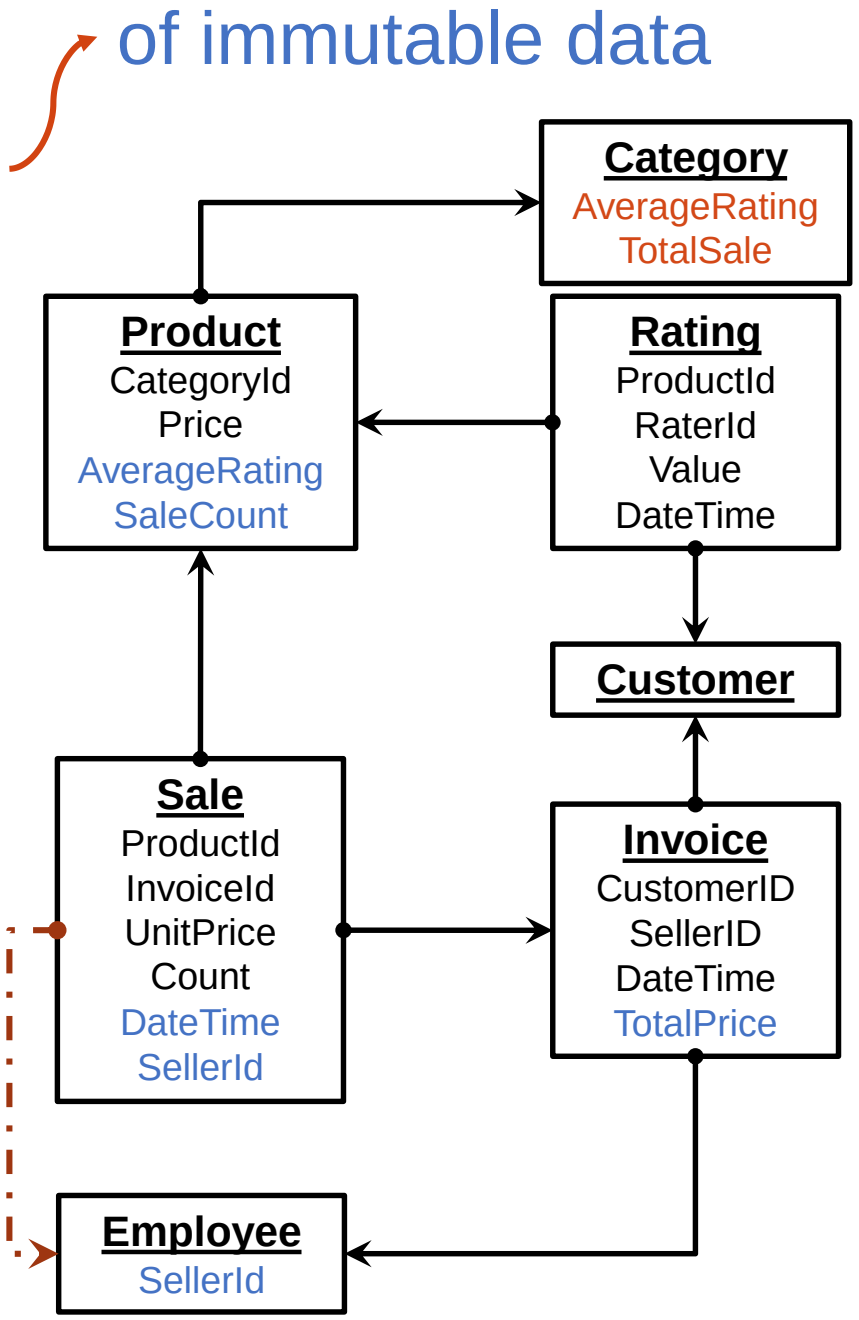


Which tables need to be joined for the following use cases?

1. A customer wants to see top rated products
2. A sales manager wants to find products with most number of sales
3. A sales manager wants to find products with most sale amount last month
4. A sales manager wants to award the top 3 salespersons of the last month
5. A marketing manager wants to award discount coupons to customers who purchased more than 10,000 Taka last month
6. A customer wants to see popular categories
7. Top management want to see a chart showing total sale of top 5 categories in a May 2020
8. A sales manager wants to see salespersons' sale per category

Speed up the queries by redundancy

1. A customer wants to see top rated products (Previously Product, Rating)
2. A sales manager wants to find products with most number of sales (Previously Product, Sale)
3. A sales manager wants to find products with most sale amount last month (Previously Product, Sale, Invoice)
4. A sales manager wants to award the top 3 sales persons of the last month (Previously Sale, Invoice, Employee)
5. A marketing manager wants to award discount coupons to customers who purchased more than 10000 Taka last month (Previously Customer, Invoice, Sale)
6. A customer wants to see popular categories (Previously Category, Product, Rating)
7. Top management want to see a chart showing total sale of top 5 categories in a May 2020 (Previously Sale, category, product, Invoice, rating)
8. A sales manager wants to see sales persons' sale per category (Previously Employee, Invoice, sale, Product, Category)



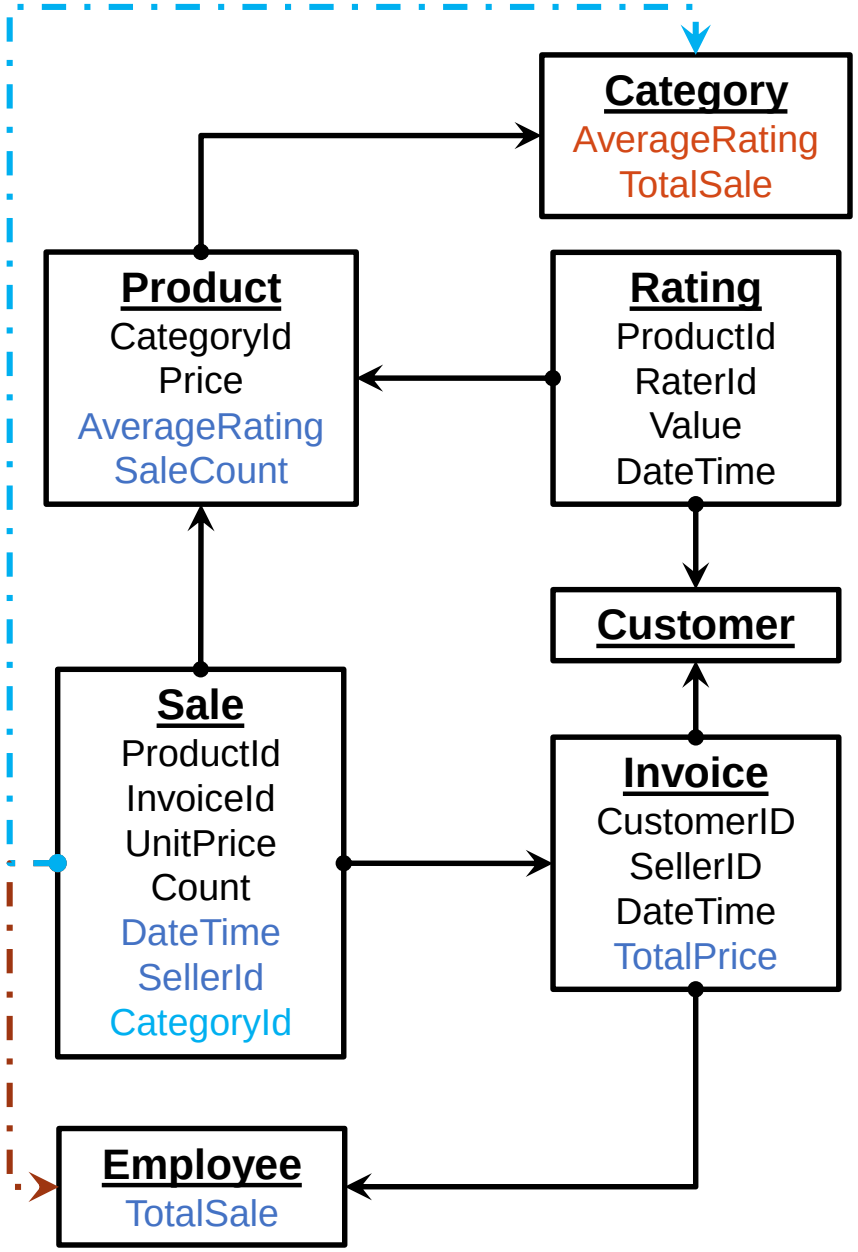
What if we keep a categoryId in Sale?

Redundancy of mutable data

- Wrong updates can be done from different parts of the code
  - Solution: Make sure update of one data is done by only one piece of code
- Updates can be slow
  - No big deal if update is not so frequent

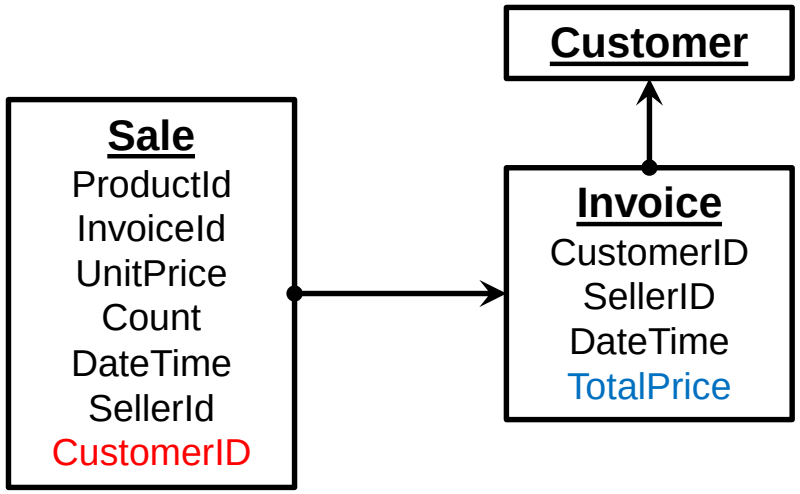
Lets solve now

7. Top management want to see a chart showing total sale of top 5 categories in a May 2020 (Previously Category, product, Sale)
8. A sales manager wants to see sales persons' sale per category **(**Previously Employee, Sale, Product, Category**)**



Choices need to be made

- A marketing manager wants to award discount coupons to customers who purchased more than 10,000 Taka last month (Customer, Invoice, Sale)

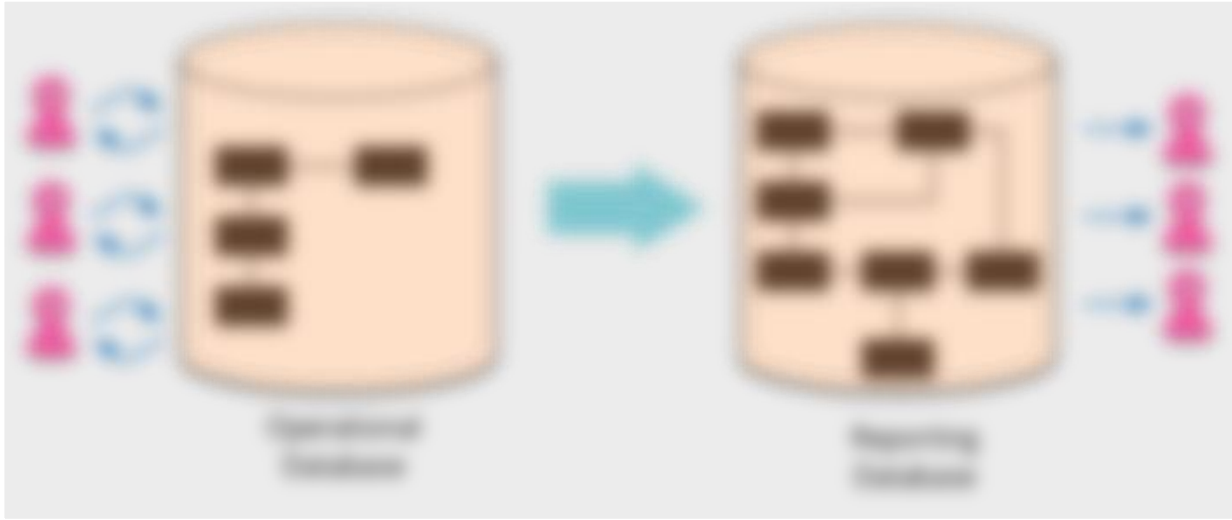


So, what did we gain by using redundancy? advantage of denormalization

- Faster query
- Easier query
- Meaningful query, examples
  - A customer wants to see top rated products
    - queries just product
  - A customer wants to see popular categories
    - queries just Category
  - Top management want to see a chart showing total sale of top 5 categories in a May 2020
    - queries Sale and Category
  - A sales manager wants to see sales persons' sale per category
    - queries Employee, Sale, Category

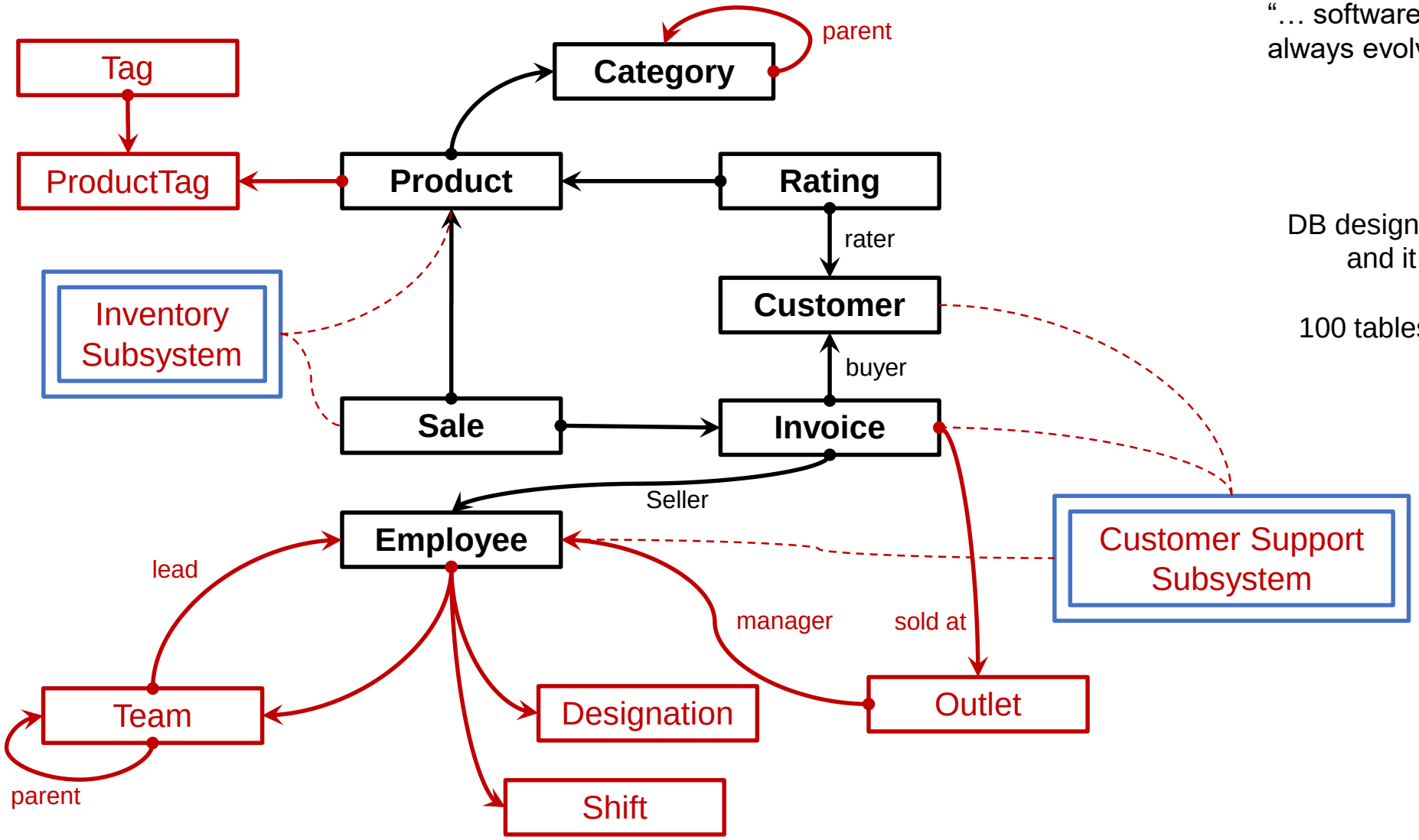
Reporting Database

Software Design and Architecture (S



Title Image Source: https://martinfowler.com/bliki/ReportingDatabase.html

More on the kid's shop DB



"…software designs tend to be incredibly large and complex."

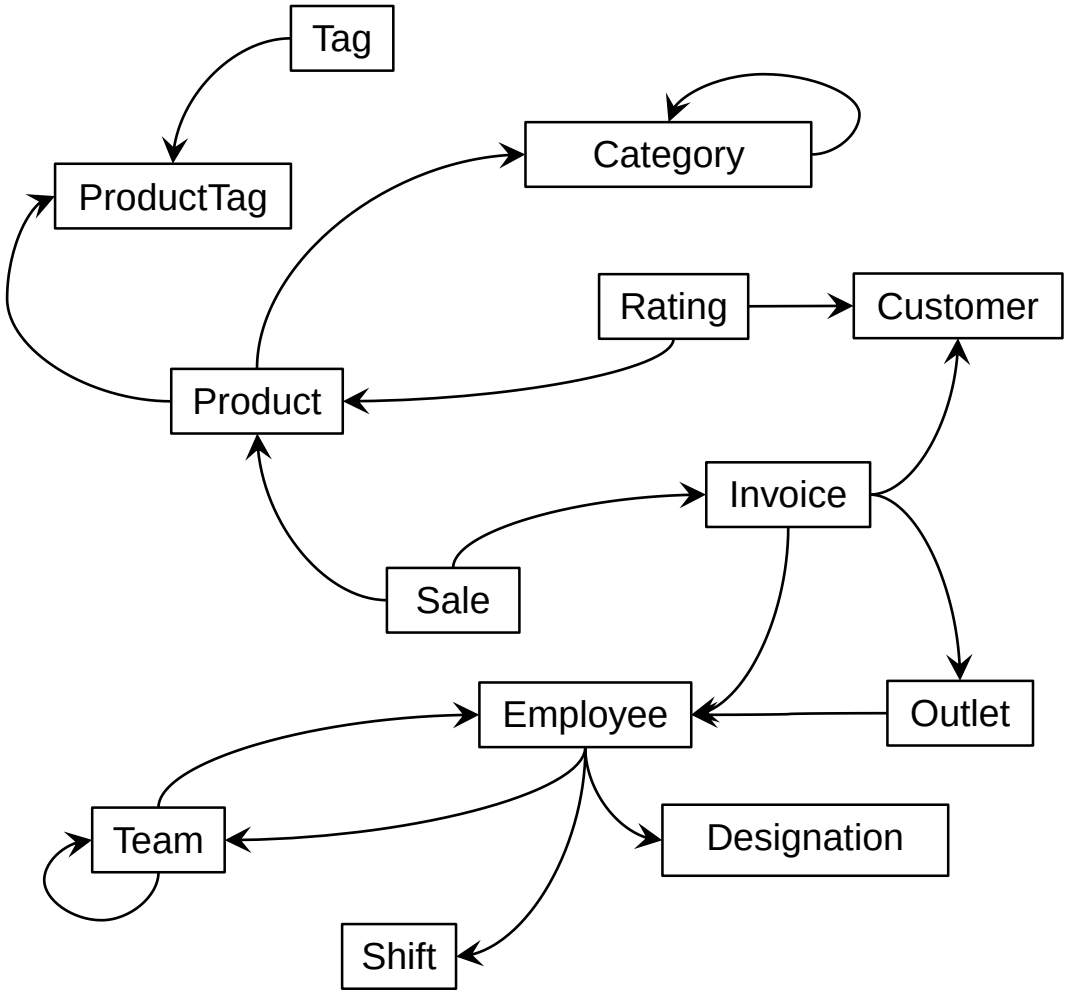
"… software designs are almost always evolving."

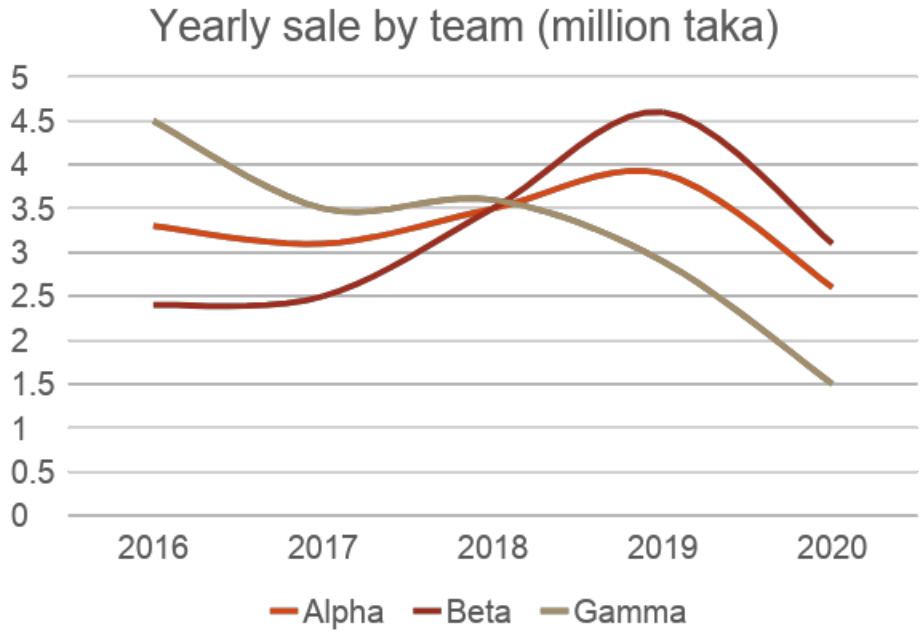
- Jack Reeves

DB design is part of the software design, and it also grows rather rapidly.

100 tables are many, but not too many.

Reporting queries





Managers want to know

- Which categories are popular in which outlets?
- Which shift can be served with fewer employees?
- Are some employees more productive than others?
- Which categories are no longer selling well?
- Which salespersons are good at selling not-so-popular products?
- Which product is not selling well but are filling the storage – run a promotion?

We want to keep normalization for consistent and fast update operation, we also want fast query

Normalization and fast query does not just work together

Attribute redundancy helps, but gets complicated over time

Redundant data means more storage
Inconsistency, You have to sync changes

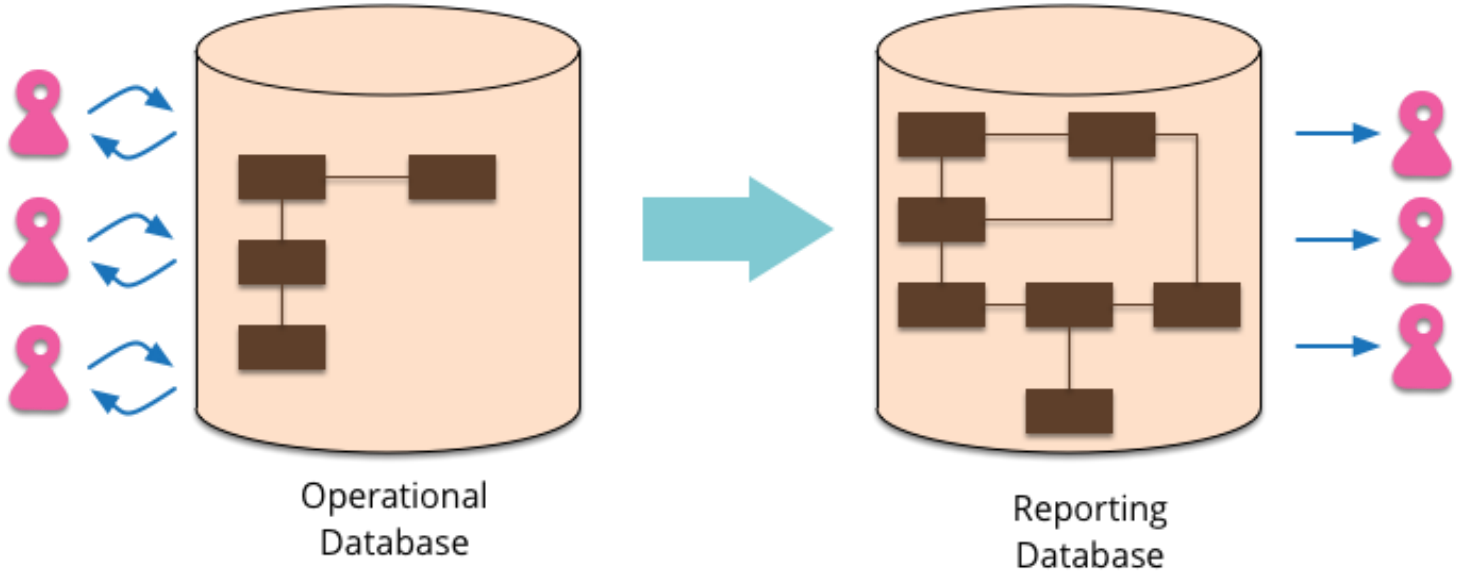
How to solve this problem?

Keep normalized OLTP (Online Transaction Processing) database for updates and transactions - operational db
Build denormalized OLAP(Online Analytical Processing) for analytics- reporting db

Why not keep copies of normalized data in denormalized table(s)?

Introducing Reporting Database

Def: the formatted result of database queries and contains useful data for decision-making and analysis.



Use separate DB for operations and reports
Build the report DB from Operational DB

https://martinfowler.com/bliki/ReportingDatabase.html

Benefits of Reporting Database

- The structure of the reporting database can be specifically designed to make it easier to write reports.
- You don't need to normalize a reporting database, because it's read-only. Feel free to duplicate data as much as needed to make queries and reporting easier.
- The development team can refactor the operational database without needing to change the reporting database.
- Load of query and update are separated between reporting and operational database.
- You can store derived data in the database for easier report generation
- You may have multiple reporting databases for different reporting needs.

A case study at Streams Tech

Situation

- 50+ tables
- Moderate data volume
- Very dynamic reporting, requiring almost all tables to be joined
- Some joins yield millions of records
- Result: slow/nonresponding reports

Solution

- Use a reporting table (in same database)
- Keep track of data updates
- Schedule data synchronization
- Result: smooth reports
- Schema: star

How to sync reporting database

Option 1: Scheduled update

- Daily, hourly or may be every 10 minutes
- Pro: Easy to manage
- Con: backdated data

Option 2: Messaging

- Send message to reporting DB when there is an update in operation DB
- Pro: immediate update
- Con: difficult to manage

Reporting database terminologies

- Measure/fact: the data that we are interested about.
  - Example: selling price or sale.
- Fact table: the table that contains the fact/measure.
  - Example: the sale table

A fact table aggregates metrics, measurements or facts about business processes.

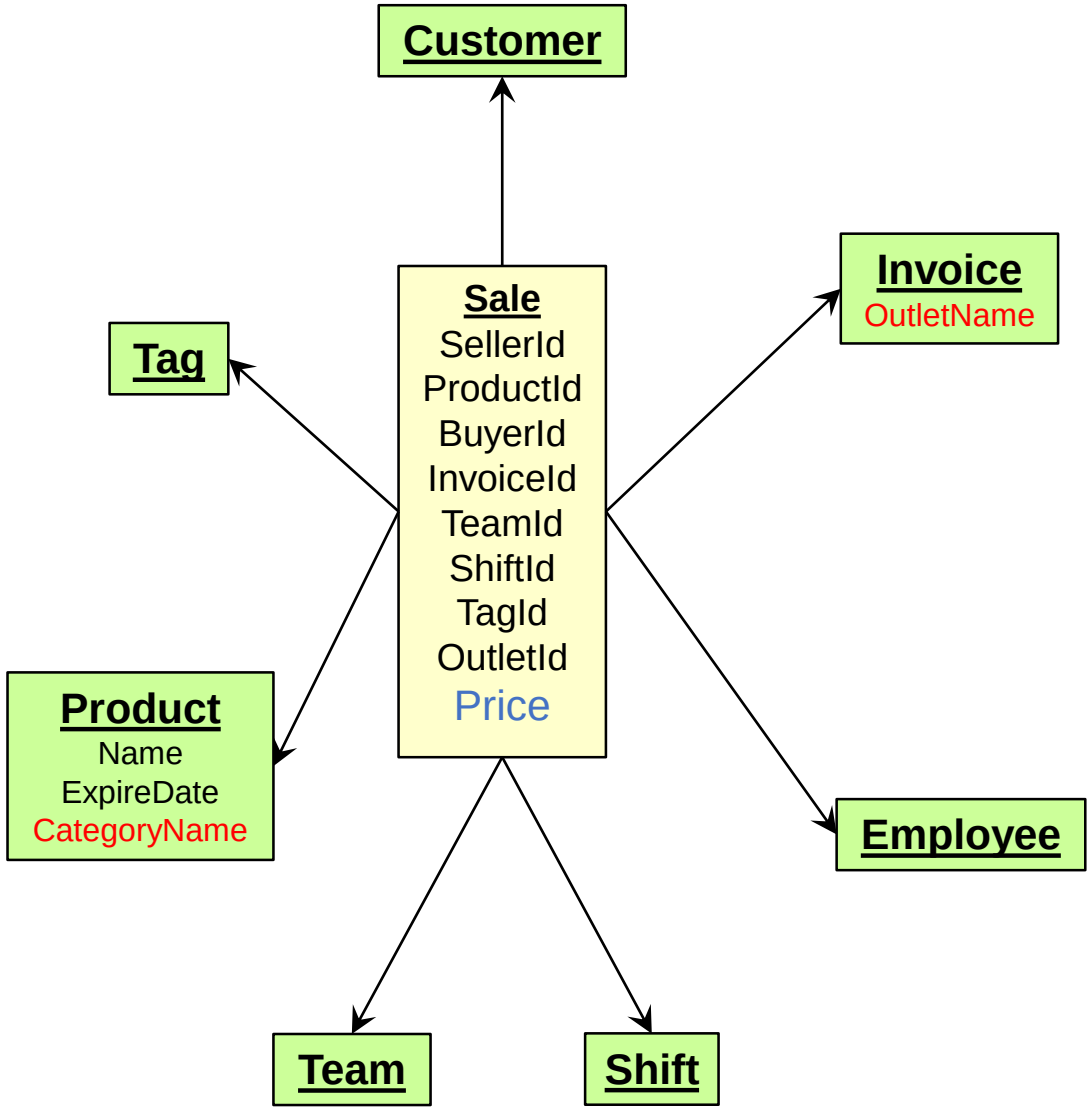
- Dimension: categories of fact.
  - Example: product
- Dimension table: the table that contains the dimension.
  - Example: product table

https://www.softwareag.com/en_corporate/blog/streamsets/schemas-data-warehouses-star-galaxy-snowflake.html

Star Schema

the center of the star can have one fact table and a number of associated dimension tables.

- Dimension tables surrounding a fact table
- One table per dimension
- Dimension table contains the set of attributes.
- The dimension table is joined to the fact table
- The dimension table are not joined to each other
- Fact table contain key and measure
- The dimension tables may not be normalized
- The dimension table is joined to the fact table using a foreign key



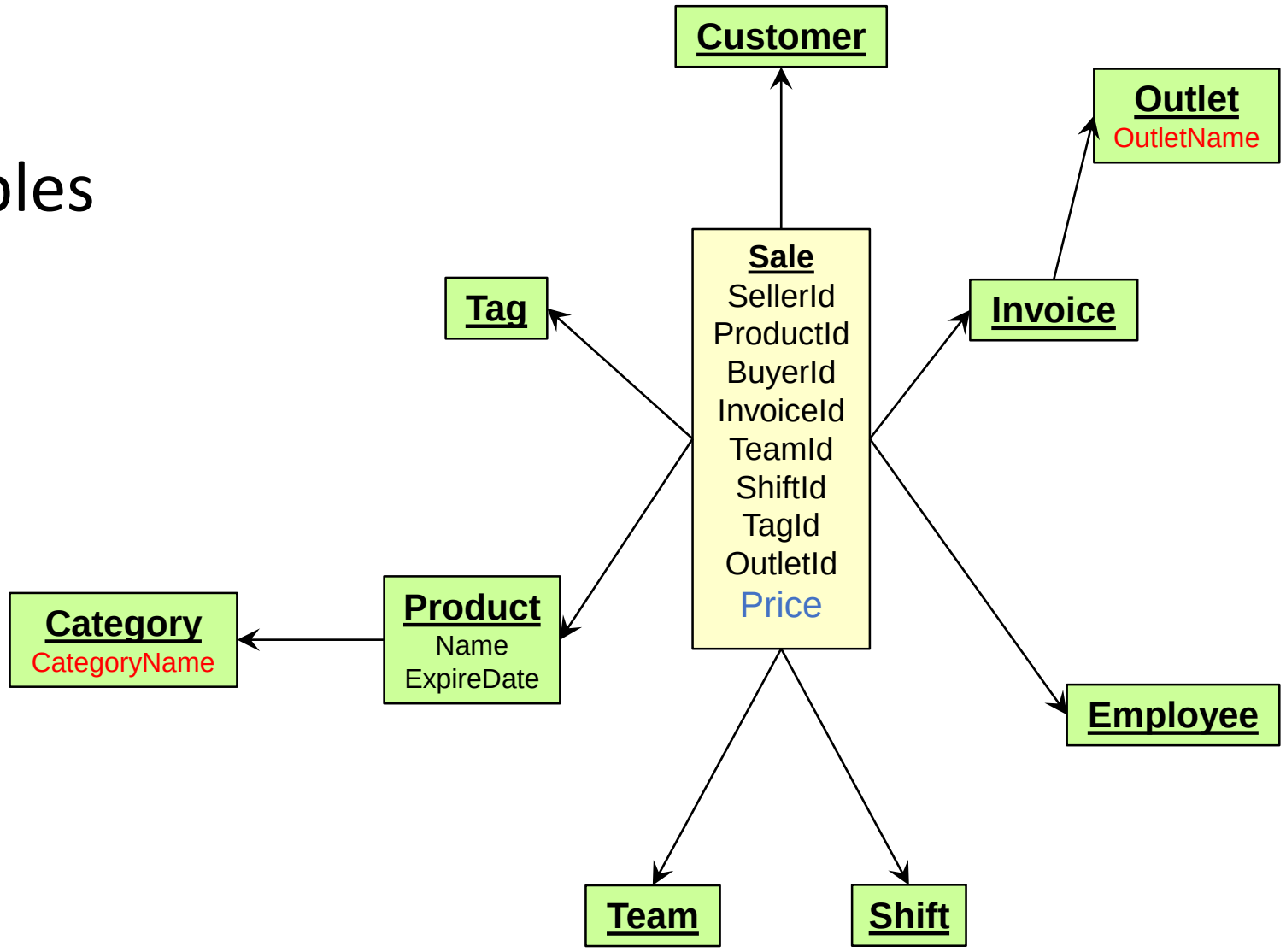
https://www.guru99.com/star-snowflake-data-warehousing.html

Snowflake Schema

Normalized dimension tables

Characteristics of Snowflake Schema:

- The main benefit of the snowflake schema it uses smaller disk space.
- Easier to implement a dimension is added to the Schema
- Due to multiple tables query performance is reduced
- (primary challenge) you need to perform more maintenance efforts because of the more lookup tables.



qs: https://www.geeksforgeeks.org/fact-constellation-in-data-warehouse-modelling/

Star vs Snowflake

	Star Schema	Snowflake Schema
Normalization	Pure denormalized dimension tables	Have normalized dimension tables
Maintenance	More redundancy due to denormalized format so more maintenance required	Less redundancy so less maintenance
Query	Simple queries due to pure denormalized desing	Complex queries due to normalized dimension tables
Joins	Less joins	More joins
Usage Guidelines	More than data integrity, speed and performance is concern here	Concerned about data integrity and duplication

Galaxy Schema

A **Galaxy Schema** contains two fact table that share dimension tables between them

Characteristics of Galaxy Schema:

- The dimensions in this schema are separated into separate dimensions based on the various levels of hierarchy.
- For example, if geography has four levels of hierarchy like region, country, state, and city then Galaxy schema should have four dimensions.
- Moreover, it is possible to build this type of schema by splitting the one-star schema into more Star schemes.
- The dimensions are large in this schema which is needed to build based on the levels of hierarchy.
- This schema is helpful for aggregating fact tables for better understanding.